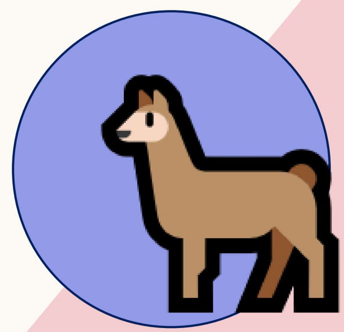
Overview

Pretraining -> GPT3

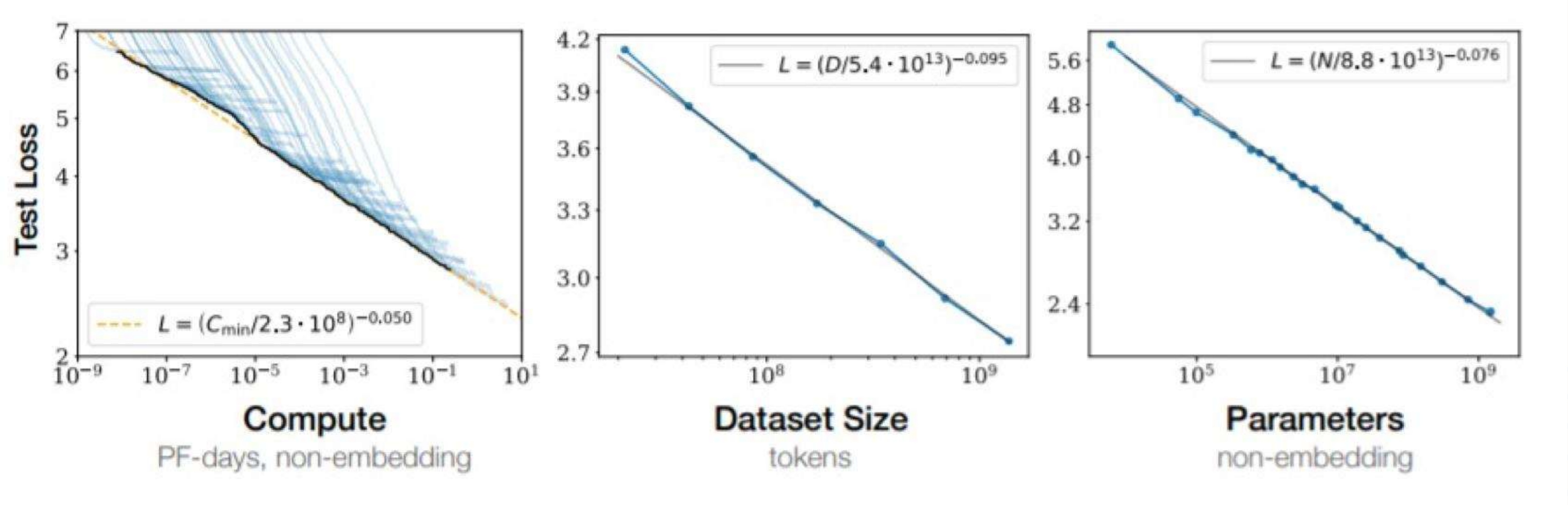
- Task & loss
- Evaluation
- Data
- Scaling laws

Post-training -> ChatGPT



Scaling laws

- Empirically: more data and larger models => better performance
  - Large models =/> overfitting
- Idea: predict model performance based on amount of data & parameter



It works for many things!

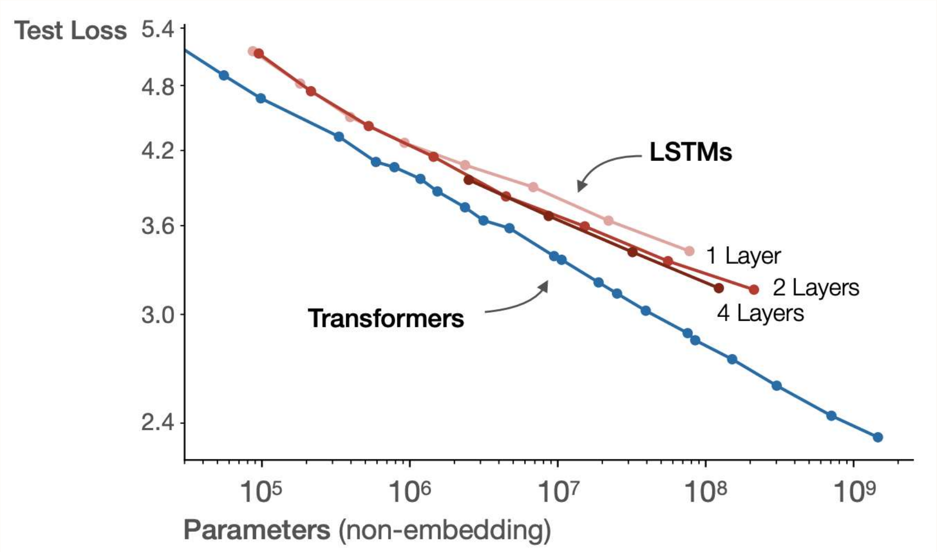
Scaling laws
[Kaplan+ 2020]

Scaling laws: tuning

- You have 10K GPUs for a month, what model do you train?
- Old pipeline:
  - Tune hyperparameters on big models (e.g. 30 models)
  - Pick the best => final model is trained for as much as each filtered out ones (e.g. 1 day)
- New pipeline:
  - Find scaling recipes (eg lr decrease with size)
  - Tune hyperparameters on small models of different sizes (e.g. for <3 days)
  - Extrapolate using scaling laws to larger ones
  - Train the final huge model (e.g. >27 days)

Scaling laws: eg LSTM

- Q: Should we use transformers or LSTM?

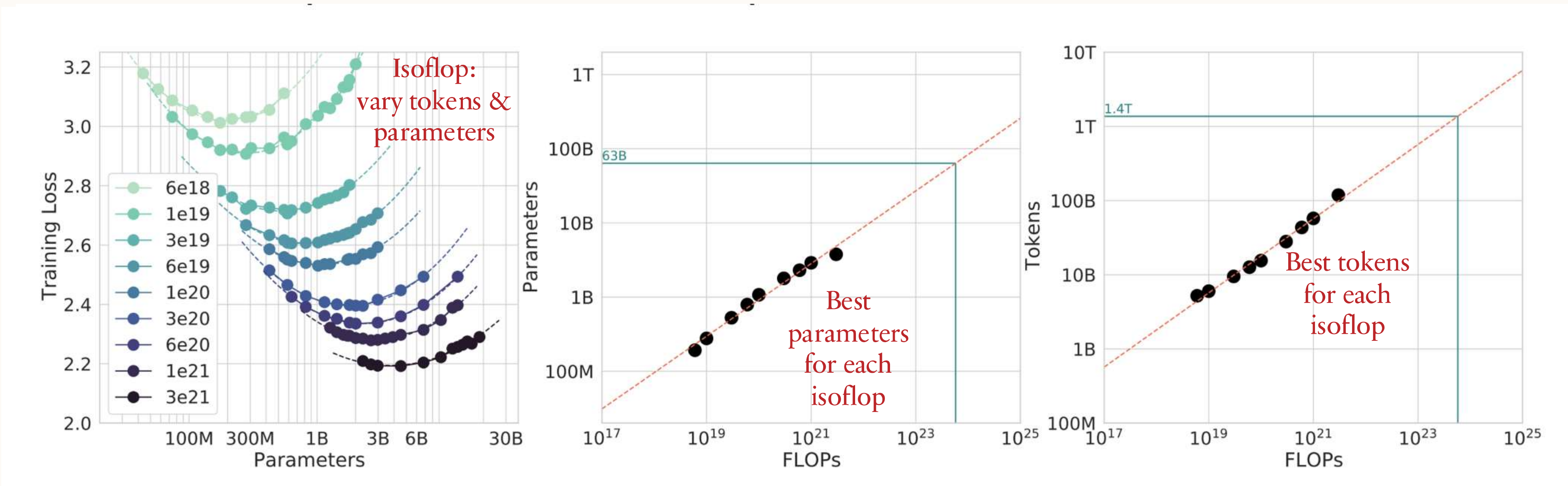


A: Transformers have a better constant and scaling rate (slope)

Scaling laws
[Kaplan+ 2020]

Scaling laws: eg Chinchilla

- Q: How do we optimally allocate training* resources (size vs data)?



A: Use 20:1 tokens for each parameter (20:1)

*doesn't consider inference cost => in practice use larger (> 150:1)

Chinchilla
[Hoffmann+ 2022]

Scaling laws: tuning

- Many questions you can try to answer with scaling laws
- Resource allocation:
  - Train models longer vs train bigger models?
  - Collect more data vs get more GPUs?
- Data:
  - Data repetition / multiple epochs?
  - Data mixture weighting?
- Algorithm:
  - Arch: LSTMs vs transformers?
  - Size: width vs depth?

Bitter lesson

- **Bitter lesson:** models improve with scale & Moore's Law

  => "only thing that matters in the long run is the leveraging of computation."

  Bitter [Sutton 2019] http://www.incompleteideas.net/IncIdeas/BitterLesson.html

- Don't spend time over complicating: do the simple things and scale them!

Training a SOTA model

- Example of current SOTA: LLaMA 3 400B

~40 tok/param => train compute optimal

Data: 15.6T tokens

Parameters: 405B

- FLOPs: 6NP = 6 * 15.6e12 * 405e9 = 3.8 e25 FLOPs

~2x less than executive order

- Compute: 16K H100 with average throughput of 400 TFLOPS
- Time: 3.8e25 / (400e12 * 3600) = 26M GPU hour / (16e3 * 24) = 70 days

From paper: ~30M

- Cost: rented compute + salary=~$2/h*26Mh + 500k/y*50employee= $52M+$25M = ~$75M

$65-85M

- Carbon emitted= 26Mh*0.7kW*0.24kg/kWh = 4400 tCO2eq

~2k return tickets JFK-LHR

- Next model? ~10x more FLOPs